

IT7075 · APPLIED AI FOR CYBERSECURITY · MODULE 3

LLMs on Local & Cloud

One model idea, three places to run it — a hosted API, a GPU you rent for a session, and a GPU you keep for the term.

Four parts · two hands-on labs · one fair comparison

Textbook: Santos — hosting & running LLMs

PART 1

Concepts & the Landscape

The vocabulary, the three places a model can run, and the reframe that makes the choice obvious: who controls the GPU?

MODULE MAP

Concepts & the Landscape

 The API and a Rented GPU

 A GPU You Keep

 Labs, Ethics & Recap

The vocabulary, fast

Five words. The rest of the module leans on all of them.

TOKENS — THE UNIT OF EVERYTHING



9 tokens, 6 words — an IP address costs more than you would guess

Term	What it means, and why you care
Tokens	The chunks of text a model reads and writes. APIs bill per token, so tokens are cost
Context window	How much text it can consider at once — its entire short-term memory
Parameters	The learned numbers. "7B" is seven billion of them. This determines the memory it needs
Quantization	Storing those numbers in fewer bits so a big model fits a small GPU
Inference	Running the model to generate output. This is the part that uses the GPU

The one relationship to memorise

Parameters determine memory.
Quantization reduces bytes per parameter.
Context window consumes memory on top.

Everything in Part 2 is that sentence with numbers in it.

Model size	Roughly
7-8B	A capable general model
14B	Noticeably better reasoning
70B+	Beyond a single consumer GPU

Three ways to run an LLM

Ways 2 and 3 are the same skill applied in two places. Learn it once.

1 Cloud API

OpenAI · Anthropic

Someone else's model on someone else's GPU. Most capable, least setup.

Cost Per token

Privacy Prompt leaves you

Persists Stateless

2 A GPU you rent

Colab T4

You self-host an open model on a remote GPU, for one session.

Cost Free

Privacy Stays on the machine

Persists Resets each session

3 A GPU you keep

Your NAIRR VM

The same open model, the same way — on hardware that is yours all term.

Cost Free

Privacy Stays on the machine

Persists Persists all term

Ways 2 and 3 are the SAME skill — self-hosting an open model on a remote GPU

Why that matters for how you study this module: the notebook that loads a model on a Colab T4 is the same notebook that loads it on your VM. Open it over Remote-SSH, pick the VM's kernel, and the cells do not change. You are not learning two techniques — you are learning one and choosing where to point it.

Three ways, five axes

The same security task, run three ways. Fill this table in yourself during the lab — it is 15 of its marks.

	WAY 1 • CLOUD API	WAY 2 • COLAB T4	WAY 3 • NAIRR VM
Cost	per token, forever	free tier	free via allocation
Privacy	prompts leave your network	private-ish	fully private
Persistence	stateless	resets every session	persists all term
Capability	highest available	good — about 7-8B	good — 14B and up
Speed	fast	fast	fast

“Self-hosting” does not mean “on my laptop.” It means on a GPU you control. That is the most common misconception in this module according to the solutions key — and Part E of the lab is exactly this table, in your own words, for 15 marks.

The reframe: who controls the GPU?

"Local vs cloud" is the wrong axis. It puts your VM on the wrong side of the line.

THE NAIVE SPLIT — AND WHY IT MISLEADS

"Local means my laptop. Cloud means someone else's computer."

By that test your NAIRR VM is "cloud" — and therefore not private. That conclusion is wrong.

THE TWO QUESTIONS THAT ACTUALLY DECIDE IT

Your data leaves your control

Cloud API (OpenAI, Anthropic)

a public service — your prompt is sent to a third party

Fine for synthetic data and course labs. Not fine for real logs, PII, or anything under a client agreement.

Your data stays on a machine you control

Colab T4

remote, but yours for the session

Your NAIRR VM

remote, and yours all term

Both are remote hardware. Both are private. "Remote" and "exposed" are not the same property, and conflating them is the mistake this slide exists to prevent.

Compare on five axes

Read it by row. Each row is one trade-off — and no column is green all the way down.

Axis	Cloud API	Colab T4 (rented)	NAIRR VM (kept)
Cost	Per token — every experiment has a price	Free to you	Free to you
Privacy	Prompt goes to a third party	Stays on the machine	Stays on the machine
Persistence	Stateless — remembers nothing	Resets every session	Persists all term
Capability	Highest — frontier models	Good: 7–8B on a T4	Good: 14B+ on an A100 slice
Speed	Fast	Fast	Fast

Look at the shape, not the cells. No column is green all the way down. The API is the only one green on capability — and the only one that is not green on privacy. The VM comes closest to sweeping, and it still concedes capability. There is no correct answer here; there is a correct answer for a particular constraint, and part of what the lab report assesses is whether you identified which constraint your task actually had.

"Can I just use the API for everything?"
Yes — until the data is sensitive, or the bill is yours. Those are the two rows that stop you, and in professional work they stop you often.

This module's files and reading

Two notebooks, two labs, one chapter. Here is what each is for.

The repository for this module

code/

host_LLM_GPU.ipynb

llm_compare.ipynb

assignments/

lab_a_three_ways/

lab_b_private_ai/

forum.md

transformers + 4-bit; runs on Colab OR your VM

Ollama on the VM; two models, one prompt

the main lab, plus a sample solution

the NAIRR deep dive

the discussion topic **and** what a good post does

Start with	If you
The primer video	Found the vocabulary slide unfamiliar
host_LLM_GPU.ipynb	Want to get a model running today
The Santos chapter	Want the reasoning behind the choices

Reading — Santos, Agentic AI for Cybersecurity

The chapter on hosting and running LLMs. Read it alongside host_LLM_GPU.ipynb rather than before — the chapter explains why the notebook does what it does, and the pairing works better than either alone.

Same notebook, two targets: host_LLM_GPU.ipynb is written so the cells do not change between Colab and your VM. Only the kernel you select changes.

Model versions move fast. Pin what you use; record it in the report.

PART 2

The API and a Rented GPU

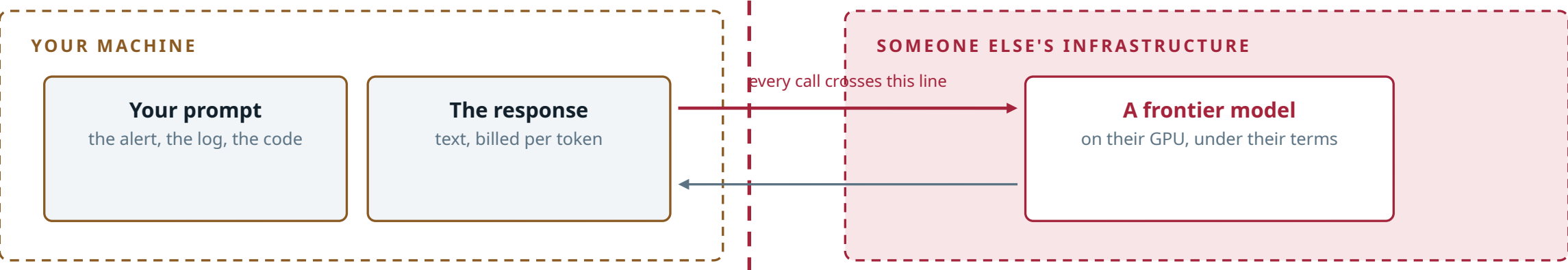
Way 1 as the capability baseline, the quantization arithmetic that makes free-GPU hosting possible, and Way 2 on a Colab T4.

MODULE MAP

-  Concepts & the Landscape
-  **The API and a Rented GPU**
-  A GPU You Keep
-  Labs, Ethics & Recap

Way 1 — the cloud API, as your baseline

You have already done this. Today it becomes the yardstick, not the destination.

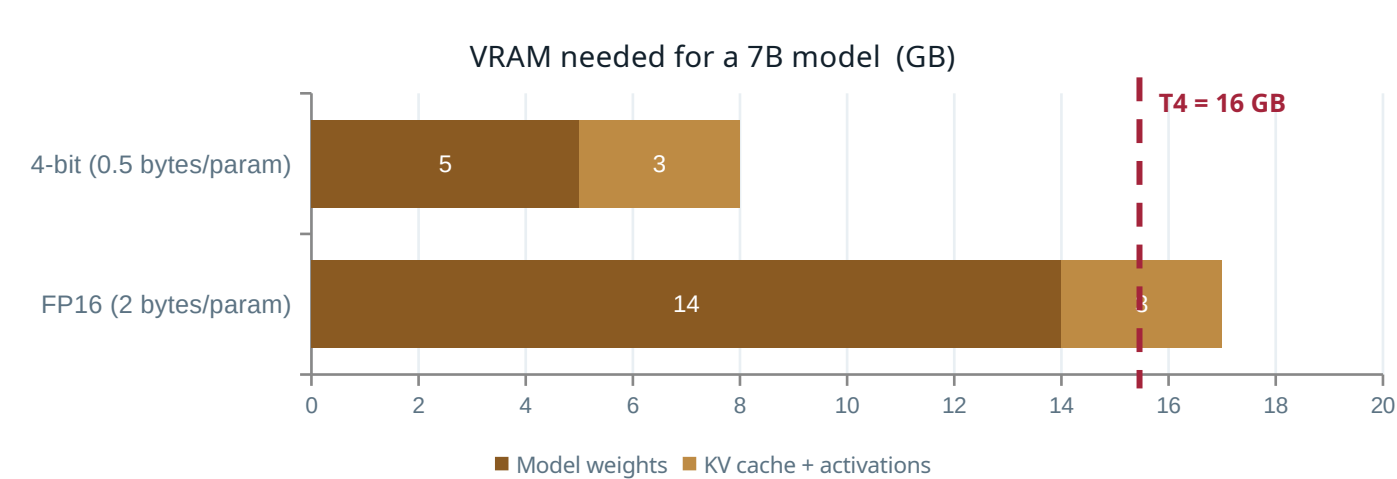


What you get	What it costs	Why it is the baseline
Highest capability, least setup	Per token — a sloppy loop is an expensive loop	"How close did the self-hosted model get?" needs a reference point
No install, no GPU, no driver versions	Every prompt lands on third-party infrastructure	Same prompt, three runners — keep the prompt you use here
A few lines of code	Nothing persists between calls	It is the ceiling, not the recommendation

Keep the key in .env or Colab Secrets. Not in a cell, and not in a screenshot in your lab report. I have seen keys leak both ways.

Fitting a big model on a small GPU

A 7B model at full precision does not fit a T4 — and the failure is confusing when it arrives.



The top bar crosses the line. 17 GB needed, 16 GB available. You get a CUDA out-of-memory error about four tokens into generation — which is a confusing place to fail, because loading appeared to succeed.

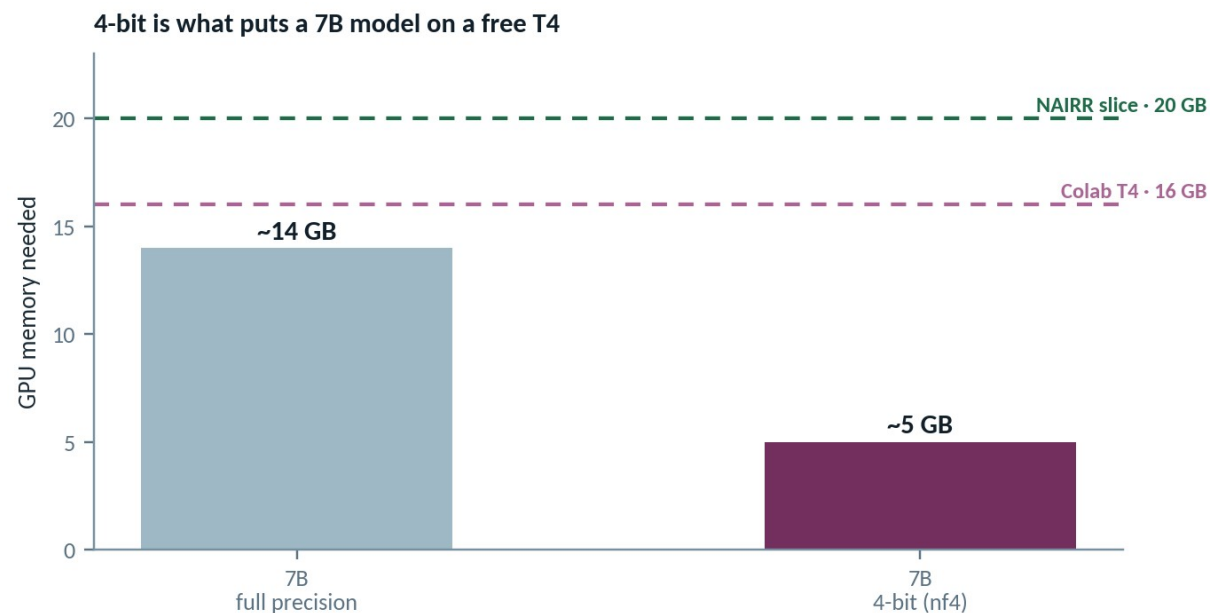
Precision	Bytes	7B needs
FP16	2	14 GB
8-bit	1	7 GB
4-bit	0.5	~5 GB
2-bit	0.25	research only

Memorise the multiplication, not the examples:
params (B) × bytes-per-param = GB of weights. Then add 2–3 GB. That tells you whether a model fits any GPU you are ever handed.

Tool	Where	The cost you pay
bitsandbytes	In transformers — roughly one config flag	A couple of percent on hard benchmarks; invisible on these labs
GGUF	In Ollama and llama.cpp — baked into the file you pull	Same trade: small accuracy hit, four-fold memory win

How a 7B model fits on a free T4

Quantisation is the whole trick. Four bits per weight instead of sixteen, and the model drops from ~14 GB to ~5 GB.



host_LLM_GPU.ipynb — THE THREE LINES THAT MATTER

```
bnb = BitsAndBytesConfig(  
    load_in_4bit=True,  
    bnb_4bit_quant_type="nf4",  
    bnb_4bit_compute_dtype=torch.float16)  
  
model = AutoModelForCausalLM.from_pretrained(  
    MODEL_ID, quantization_config=bnb,  
    device_map="auto")
```

Storage and compute precision are separate. The weights live in 4 bits; the arithmetic still runs in float16. nf4 is a format shaped to how weights are actually distributed, which is why the quality cost is small.

If it will not fit, it is one line

MODEL_ID = "Qwen/Qwen2.5-3B-Instruct" — the notebook names this fallback explicitly.

Hosting a model, end to end

The core of `host_LLM_GPU.ipynb`: load a quantised model, then talk to it through the chat template.

host_LLM_GPU.ipynb — CELLS 17 AND 19

```
!pip install -q -U transformers accelerate bitsandbytes

import torch
from transformers import (AutoModelForCausalLM, AutoTokenizer,
                          BitsAndBytesConfig)

MODEL_ID = "Qwen/Qwen2.5-7B-Instruct"  # fits a T4 in 4-bit

bnb = BitsAndBytesConfig(load_in_4bit=True,
                        bnb_4bit_quant_type="nf4",
                        bnb_4bit_compute_dtype=torch.float16)
tok = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, quantization_config=bnb, device_map="auto")
print("Loaded", MODEL_ID, "on", model.device)

def chat(prompt, system="You are a concise cybersecurity assistant.",
        max_new_tokens=256):
    messages = [{"role": "system", "content": system},
                {"role": "user", "content": prompt}]
    ids = tok.apply_chat_template(messages,
                                add_generation_prompt=True,
                                return_tensors="pt").to(model.device)
    out = model.generate(ids, max_new_tokens=max_new_tokens,
                       do_sample=False)
    return tok.decode(out[0][ids.shape[-1]:],
                    skip_special_tokens=True)
```

1 device_map="auto"

Puts the model on the GPU if there is one. If the print says `cpu`, stop — everything after will take minutes per answer.

2 apply_chat_template is not optional

Every instruct model has its own turn-marking format. The tokenizer knows its own; hand-rolling the string costs quality silently.

3 Decode only the new tokens

`out[0][ids.shape[-1]:]` drops the prompt. Without the slice you print your own question back.

4 do_sample=False

Greedy decoding — same input, same output. Required if you want the comparison in Part D to mean anything.

The upgrade that breaks the notebook

``pip install -U transformers`` pulls 5.x, where one default flipped — and the failure surfaces two lines later.

``pip install -U transformers`` is in the notebook. That `-U` is the whole problem.

transformers 4.x — what the notebook was written for

`apply_chat_template(..., return_dict=False)`

```
ids = tok.apply_chat_template(
    messages, return_tensors="pt")

ids.shape[-1]      -> 42
model.generate(ids, ...) -> works
```

a **Tensor** — everything downstream is fine

transformers 5.x — what a 2026 student installs

`return_dict` now defaults to `True`

```
ids = tok.apply_chat_template(
    messages, return_tensors="pt")

ids.to(device)      -> still works (!)
ids.shape[-1]       -> AttributeError
```

a **BatchEncoding** — and `.to()` masks it until later

One-kwargs fix: `return_tensors="pt", return_dict=False` — or unpack with `**ids` and read `ids["input_ids"].shape[-1]`

The error is not where the problem is. `.to()` accepts the wrong type quietly, so the failure surfaces two lines later on a line that looks innocent. That pattern — a permissive method hiding a type change — is worth naming; you will meet it again.

Pin your versions

A tutorial that says ``-U`` has a shelf life. This affects the core teaching cell of Lecture 2 and graded Parts B and D of the lab.

Way 2 — host the model on a Colab T4

The same notebook runs on your VM. Only the kernel changes.

```
host_LLM_GPU.ipynb — Runtime ► Change runtime type ► T4 GPU

from transformers import AutoModelForCausalLM, AutoTokenizer
from transformers import BitsAndBytesConfig
import torch

MODEL = "Qwen/Qwen2.5-7B-Instruct" # Apache-2.0, not gated

qcfg = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
)

tok = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForCausalLM.from_pretrained(
    MODEL, quantization_config=qcfg, device_map="auto")

def chat(prompt, max_new_tokens=256):
```

Choice	Why this one
Qwen2.5-7B-Instruct	Apache-2.0 and ungated — nobody gets stuck on an access token
load_in_4bit	~5 GB instead of 14 — it fits with headroom
nf4	The quantization type tuned for normally-distributed weights
device_map="auto"	Puts the layers on the GPU without you enumerating them

Three front doors on the same VM: a Python call, a ! shell cell, or a terminal. Same machine underneath.

Then run it on your VM without changing a cell: open the same notebook over VS Code Remote-SSH and select the VM's kernel. That is the whole difference between Way 2 and Way 3 at the code level — which is the point I made in Part 1, now made concrete.

Shop the Hugging Face Hub

Choosing a model that fits your hardware and your task is a hireable skill.

1 Filter

Task ► Text Generation. Sort by trending or downloads to see what people actually run.

2 Read the card

Three things: parameter count, licence, and whether access is gated.

3 Check it fits

Params × bytes-per-param + 2–3 GB. Do this before you download anything.

4 Match the task

A small open model is fine for triage and extraction. Save frontier models for hard reasoning.

On the model card	Looks like	What it decides	The trap
Parameter count	7B, 14B, 70B	Whether it fits your GPU at all	Names are inconsistent — check the card, not the title
Licence	Apache-2.0, MIT, custom	Whether you may use or redistribute it	"Open weights" is not the same as "open licence"
Gated?	A request-access button	Whether you can download it today	Approval can take days — fine for a project, fatal in a lab session
Base vs Instruct	-Instruct suffix	Whether it follows instructions or just continues text	A base model will not answer your question; it will extend it

PART 3

A GPU You Keep

Self-hosting on your NAIRR VM with Ollama — a private, persistent model server you reach safely over an SSH tunnel.

MODULE MAP

- Concepts & the Landscape
- The API and a Rented GPU
- **A GPU You Keep**
- Labs, Ethics & Recap

Self-host on the VM with Ollama

One command to install. A persistent server that is still running tomorrow.

```
On the VM, over SSH

curl -fsSL https://ollama.com/install.sh | sh      # detects the GPU

ollama pull llama3.1:8b          # ~5 GB
ollama pull qwen2.5:14b         # ~10 GB

ollama list                      # what you have
ollama run llama3.1:8b          # an interactive prompt, right now

# and it is serving on localhost:11434 for your code to call
```

The difference that matters: this server does not reset. Close your laptop, come back tomorrow, and the model is still loaded and still answering. Nothing you send it ever leaves the VM.

Model	Fits the ~20 GB slice?
llama3.1:8b	Yes — ~5 GB, comfortable
qwen2.5:14b	Yes — ~10 GB, still room
qwen2.5:32b	No — falls back to CPU
llama3.1:70b	No — not on one slice

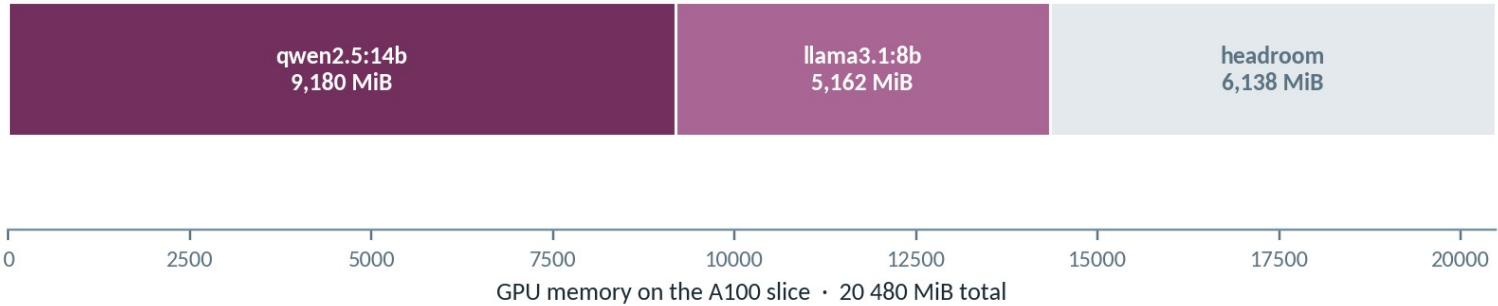
Why Ollama rather than transformers here: it manages quantization, model files and a persistent server for you. transformers works too — Ollama is the fast path.

Check it landed on the GPU: ollama ps — the PROCESSOR column should read 100% GPU.

Twenty gigabytes, and what fits in it

Two models resident at once on the A100 slice — measured with ``nvidia-smi`` and ``ollama ps``.

Both models resident, both reporting 100% GPU — 14 903 MiB of 20 480 used



MEASURED — Lab_PrivateLLM_Solutions.md REFERENCE RUN

```
$ ollama ps
NAME          SIZE    PROCESSOR  CONTEXT
qwen2.5:14b   9.9 GB  100% GPU   4096
llama3.1:8b   5.6 GB  100% GPU   4096

$ nvidia-smi
GRID A100X-20C   14903MiB / 20480MiB
llama-server    PID 69643   9180MiB
llama-server    PID 69838   5162MiB
```

Run ``ollama ps`` before you call anything slow

It tells you what is resident and, in the PROCESSOR column, whether each model is on the GPU or the CPU.

Nine times out of ten “this model is slow” means it quietly landed on the CPU — and nothing told you.

Manage the GPU — and the CPU-fallback trap

"Local LLMs are slow" is almost always a fit problem, not a speed limit.

Healthy — ollama ps

NAME	SIZE	PROCESSOR	UNTIL
llama3.1:8b	6.1 GB	100% GPU	4 minutes from now

The trap — same command, a model that did not fit

NAME	SIZE	PROCESSOR	UNTIL
qwen2.5:32b	21 GB	62%/38% CPU/GPU	4 minutes from now

No error is raised. Ollama silently spills the layers that do not fit onto the CPU and keeps working — perhaps twenty times slower. Nothing tells you except that column.

Command	What it tells you
ollama ps	What is loaded, and whether it is on the GPU
nvidia-smi	VRAM in use right now — your dashboard for what fits
ollama stop <model>	Unload it and reclaim the memory
ollama rm <model>	Delete the files from disk entirely

The fix, in order

Stop the oversized model → confirm VRAM is free in nvidia-smi → pull something at 14B or below → check ollama ps again.

Diagnose before you conclude.

Check ollama ps before deciding a model is too slow to be useful.

It is a fit problem, not a speed limit

Exercise 2.5 asks you to break it on purpose. Here is what breaking it actually looks like.

PULL A 32B MODEL ONTO A 20 GB SLICE AND EVERYTHING FALLS BACK TOGETHER

\$ ollama ps				
NAME	ID	SIZE	PROCESSOR	CONTEXT
qwen2.5:32b	9f13ba1299af	21 GB	100% CPU	4096
qwen2.5:14b	7cdf5a0187d5	9.9 GB	100% CPU	4096
llama3.1:8b	46e0c10c039e	5.6 GB	100% CPU	4096

Read the cascade

21 GB does not fit in 20 480 MiB, so qwen2.5:32b goes to CPU — and it drags the other two off the GPU with it.

The two that used to fit still would, alone.

It is a fit problem, not a speed limit.

Recover with `ollama stop --all ; sudo systemctl restart ollama` — then stay at 14B or below on this slice.

A system that silently degrades is harder to operate than one that refuses. Nothing errored. Nothing warned. The only signal is the PROCESSOR column in `ollama ps` — and you only look there if you already suspect the cause. Keep that pattern in mind: you will meet it again in module 4, where a retrieval query happily returns the nearest chunks even when none of them is relevant.

A browser chat of your own — Open WebUI

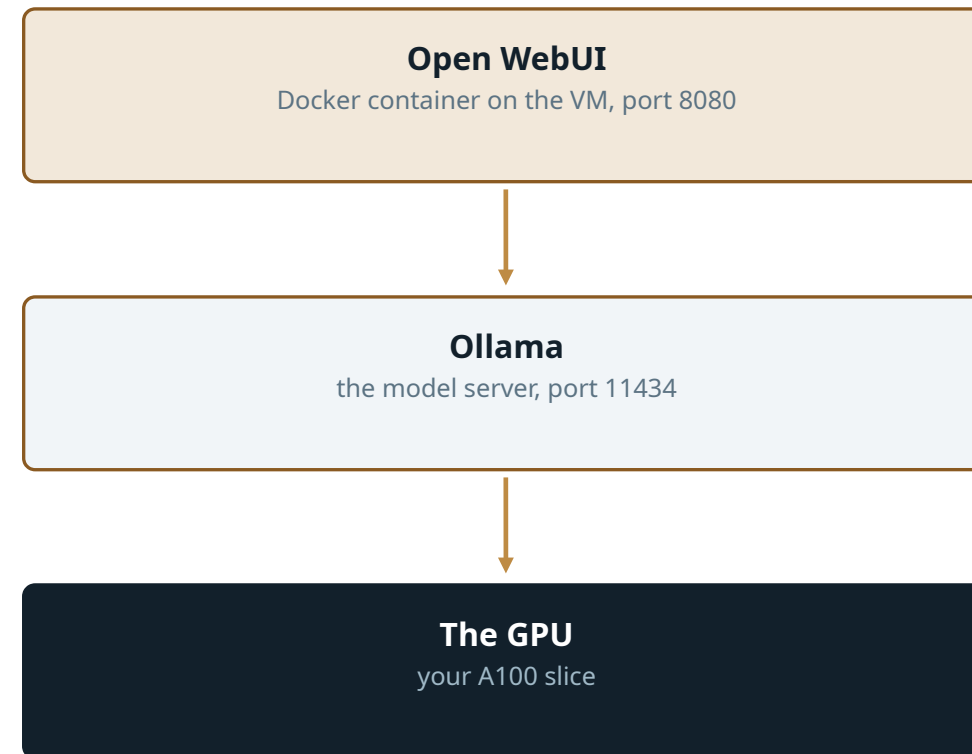
The payoff of the whole VM path: a ChatGPT-style interface that never phones home.

On the VM — bind to localhost, not to the world

```
docker run -d -p 127.0.0.1:8080:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui --restart always \
  ghcr.io/open-webui/open-webui:main
```

Note the 127.0.0.1 in that port mapping. Without it, Docker binds to every interface and your chat UI is on the public internet. This is the whole of the next slide, in one prefix.

What you now own: a private, persistent, browser-accessible assistant on hardware you control, holding conversation history that belongs to you and to nobody else.



Keep it closed, tunnel in

The security default of this module — and of every internal service you will ever run.



Port 8080 is never open to the internet. The only way in is a connection you could already make.

From your laptop

```
ssh -L 8080:localhost:8080 it7075
# then open http://localhost:8080 in your browser
```

Open the port	Tunnel instead
Anyone who scans finds it	Reachable only if you can already log in
Needs its own authentication	Inherits your SSH key

Compare the models — a fair test

Same prompt, temperature zero, same token limit, timed. Otherwise you are comparing noise.

```
llm_compare.ipynb

import ollama, time

PROMPT = open("prompt.txt").read() # the SAME one you gave the API

for model in ["llama3.1:8b", "qwen2.5:14b"]:
    t0 = time.perf_counter()
    r = ollama.chat(model=model, options={"temperature": 0,
                                         "num_predict": 256},
                    messages=[{"role": "user", "content": PROMPT}])
    print(model, round(time.perf_counter() - t0, 1), "s")
```

Hold fixed	Prompt	Temp	Max tokens
Every run	Identical	0	256

Score 1-5	Weight	What you are judging
Correctness	×3	Did it get the security question right?
Specificity	×2	Named the pattern, or hedged vaguely?
Usability	×1	Could an analyst act on it as written?
Latency	—	Recorded, not scored

The lesson it makes concrete: quality is not speed. The smaller model is faster; the larger one usually answers better.

Why the rubric matters more than the timings: without it, "which model is better" collapses into a vibe, and vibes are not defensible in a report. A crude rubric applied consistently beats a sophisticated one applied to whichever answer you happened to like. Record the numbers and submit them.

A fair test, and the one the code runs

Same prompt, temperature 0, same token budget, timed. The shipped comparison cell does none of those things.

llm_compare.ipynb — AS SHIPPED

```
def ask(model, prompt, temperature=0.7):
    t0 = time.time()
    r = requests.post(f"{OLLAMA}/api/generate", json={
        "model": model, "prompt": prompt, "stream": False,
        "options": {"temperature": temperature}})
    return r.json()["response"], time.time() - t0

for model in (MODEL_A, MODEL_B):
    text, secs = ask(model, PROMPT)    # <- 0.7, and no cap
```

WHAT THE SLIDE ACTUALLY ASKS FOR

```
def ask(model, prompt, temperature=0.0, max_tokens=200):
    r = requests.post(f"{OLLAMA}/api/generate",
        timeout=120, json={
            "model": model, "prompt": prompt,
            "stream": False,
            "options": {"temperature": temperature,
                        "num_predict": max_tokens}})
    r.raise_for_status()
    return r.json()["response"], ...
```

Deterministic is the point. At temperature 0.7 both models are sampling, so running the cell twice gives two different answers. “Qwen was more thorough” then measures a dice roll.

Measured, and counterintuitive

Temperature barely moves tokens/second — throughput is set by model size and GPU, not sampling.

No timeout, no status check: ask for a model you have not pulled and you get `KeyError: 'response'`, not a readable error — and a 32B model in CPU fallback hangs the cell with no feedback at all.

PART 4

Labs, Ethics & Recap

Two labs, the data-handling rules that apply to every prompt you send, and a look ahead to grounding models in your own documents.

MODULE MAP

- Concepts & the Landscape
- The API and a Rented GPU
- A GPU You Keep
- **Labs, Ethics & Recap**

Your week — the two labs

Lab A walks the whole arc. Lab B goes deep on the VM path.

REQUIRED

Lab A — Run an LLM Three Ways

- Re-run your API baseline and keep the prompt
- Host Qwen2.5-7B on a Colab T4 in 4-bit
- Self-host the same idea on your NAIRR VM
- Pick your own model from Hugging Face and justify it
- Compare all three on the five axes

DEEPER DIVE

Lab B — Your Own Private AI

- Install Ollama and pull two models that fit
- Prove they loaded on the GPU with `ollama ps`
- Run Open WebUI in Docker, bound to localhost
- Reach it over an SSH tunnel — port never opened
- Trigger the CPU-fallback trap on purpose, then fix it

Submit	Which means	Watch out for
Screenshots, keys hidden	Terminal and notebook output showing it ran, on your machine	A visible key means rotate it and resubmit
A short comparison report	The five axes, your rubric scores, and which you would choose for what	"It worked" is not a finding — say what you measured
A forum post	One thing that surprised you, in your own words	Post early enough that people can reply

Data handling and responsible use

The same rules as Module 1, plus one that is specific to open-weight models.

Rule	Concretely	Why it bites here specifically
Synthetic or provided data only	No real PII, secrets, credentials or live incident data in any prompt	This module sends the same prompt to a third party three times over
Treat cloud calls as retained	Assume anything sent to a hosted API could be stored and reviewed	It is third-party processing, whatever the marketing page says
Open weights still have licences	Check the model card before you use or redistribute anything	"Open" is not "unrestricted" — several major models carry real limits
Spend limits and key rotation	A hard cap in the console; rotate immediately on any leak	A loop over a dataset is exactly how an unbounded bill happens
Bind services to localhost	127.0.0.1 in the port mapping; tunnel in over SSH	An unauthenticated chat UI on a public IP is found by scanners

The one people get wrong: row three. "Open weights" means you can download and run the parameters. It does not automatically mean you may use the model commercially, fine-tune it, or redistribute a derivative. Several widely used models carry custom licences with genuine restrictions. Read the card before you build something you intend to ship.

Recap and what's next

Four things to carry forward.

1 One model, three homes

A hosted API, a GPU you rent for a session, a GPU you keep. Ways 2 and 3 are one skill.

2 Ask who controls the GPU

Not "local or cloud". Remote and exposed are different properties — your VM is remote and private.

And the honest conclusion

No option wins on all five axes. Choosing well means naming the constraint your task actually has — sensitivity, budget, persistence, or raw capability — and letting that row decide.

3 Do the VRAM arithmetic

Params × bytes-per-param, plus 2–3 GB. It is why 4-bit exists and why 32B silently crawls.

4 Keep it closed, tunnel in

Bind to localhost, forward over SSH. The right default for any internal service you ever run.

Next — RAG: you now have models running in three places, and not one of them knows anything about your organisation. Retrieval-augmented generation is how you ground them in your own documents.